

Projeto AV2, Instituição: Centro Universitário Maurício de Nassau (UNINASSAU) - Graças

Disciplina: Back-end Frameworks - 2026.1

Professor: Breno Holanda

1. O Desafio

Chegamos à reta final da nossa disciplina! A Avaliação 2 (AV2) não será uma prova escrita tradicional. Vocês participarão do **Tech Challenge UNINASSAU**, onde atuarão como verdadeiros Desenvolvedores Back-end construindo um produto real para o mercado.

O Objetivo: Desenvolver, documentar e publicar de uma API RESTful completa conectada a um banco de dados, estruturada rigorosamente no padrão **MVC**. No dia da avaliação, vocês apresentarão a solução para uma "banca de investidores" simulada.

2. Cronograma e Dedicção em Sala

O conteúdo teórico da disciplina encerrou-se. A partir do lançamento deste edital, as nossas aulas de terça-feira funcionam como um **Laboratório Prático de Mentoria**. Vocês usarão o horário da aula para programar junto com a equipe, enquanto o professor atua como *Tech Lead*, tirando dúvidas e avaliando a evolução.

- **26/05** : Divulgação deste edital, formação dos grupos, escolha do Tema e início do desenvolvimento (Foco na arquitetura MVC e Banco de Dados).
- **02/06** : Sprint final de desenvolvimento em sala. Foco na Documentação da API (Swagger), testes de requisição e preparação do Deploy na nuvem.
- **09/06** : Apresentações oficiais (Pitch de Negócios + Demonstração Técnica ao vivo).

3. Requisitos Técnicos Obrigatórios

Para que o projeto seja aceito, ele deve conter:

1. **Padrão MVC:** Separação clara de pastas e arquivos (Model, View, Controller).
2. **Persistência de Dados:** Uso de Banco de Dados com operações de CRUD (pelo menos Criar, Ler e Atualizar/Deletar).
3. **Regras de Negócio:** Tratamento de erros e validações (ex: não cadastrar itens com valores vazios, evitar matrículas duplicadas).
4. **Documentação:** A API deve ter uma documentação interativa gerada via Swagger.
5. **Deploy:** A API não pode rodar apenas no computador local. O link precisa estar acessível

na nuvem.

4. Temas Disponíveis (Escolham 1 por grupo)

A sua API deve resolver um problema de negócio focando em autonomia e baixo custo. Escolham uma das dores abaixo para solucionar:

1. **Gestão de Inventário e Alertas:** Controle de estoque para pequeno comércio com bloqueio de venda para estoque zero e alerta de reposição.
2. **Sistema de Reservas para Clínicas:** Agendamento de pacientes garantindo que não haja sobreposição de horários para o mesmo médico.
3. **API de Helpdesk / Tickets de Suporte:** Registro de chamados de TI com transição de status (Aberto -> Em Análise -> Resolvido).
4. **Gestão de Adoção de Animais:** Controle para ONGs cruzando dados de animais disponíveis, em tratamento e famílias adotantes.
5. **Assistente de Gestão Acadêmica:** Diário digital para professores cadastrarem notas e o sistema calcular automaticamente a situação do aluno.
6. **Controle de Despesas Corporativas:** Registro de gastos por funcionários que só podem ser marcados como "Pagos" por um perfil Gestor.
7. **Gestão de Frota e Manutenção:** Registro de veículos empresariais e controle de quilometragem para emitir alertas de revisão.
8. **Controle de Lotação em Eventos:** Bilheteria digital que bloqueia novos cadastros automaticamente ao atingir a capacidade máxima da sala.

5. Formato da Apresentação (Dia 09/06)

Atenção: **Não é necessário construir um Front-end (Telas/Site).** O foco é 100% no Back-end.

Cada grupo terá cerca de **15 minutos** de palco, divididos em:

- **O Pitch (5 min):** Vender a solução. Qual é a dor do cliente? Como a sua API resolve isso de forma ágil e segura? (Uso de Slides).
- **A Arquitetura (5 min):** Mostrar o código-fonte, explicando a organização no padrão MVC.
- **A Demonstração ao Vivo (5 min):** Prova de fogo! Apresentar o Swagger na nuvem e realizar requisições ao vivo (GET, POST) para provar que a API salva e retorna dados corretamente.

6. Critérios de Avaliação (Total: 10,0 pontos)

A nota da AV2 será distribuída da seguinte forma:

Critério	Descrição	Pontuação
Visão de Negócio	Clareza no pitch, entendimento do problema e postura profissional.	2,0 pts
Arquitetura (MVC)	Código limpo e devidamente separado em Model, View e Controller.	2,5 pts
Banco de Dados	Operações CRUD funcionais e tratamento de erros (ex: try/except).	2,5 pts
Documentação	Swagger bem configurado, endpoints descritivos e métodos HTTP corretos.	1,5 pts
Deploy	Aplicação publicada online e funcionando durante a demonstração ao vivo.	1,5 pts